Banking (Teller) and Shared Account Plan

Planning only. Builds on backlog VP-009/010/011 and PROJECT.md §§6–9, DESIGN.md §§14–17. Nothing here changes approved requirements; open decisions are marked **DECIDE**.

1. The "account system" already exists

Foundation's `profiles(id)` (Supabase `auth.uid()`) is the canonical user. Receipts already key on `receipts.owner_id → profiles(id)` with RLS `owner_id = auth.uid()`. Banking should do the same: `institution_connections.user_id → profiles(id)`. That single FK is the link between Receipts and Banking; no new identity model or "account service" microservice is needed (AGENTS.md §24).

```
profiles (Foundation)
 ├── receipts → items / participants / allocations   (Receipts)
 └── institution_connections → accounts → transactions (Banking)
Cross-domain rows reference profiles only, never each other (for MVP).
```

Do not link receipts to bank transactions now. Reimbursement reconciliation is explicitly outside Banking and Receipt MVP ownership. Dashboard joins both read-only at presentation time.

2. Teller and TLS

Env	Client cert (mTLS)	Data
Sandbox	Optional; validated if sent	Simulated. Default for dev/CI
Development	Required	Real bank data, opt-in only
Production	Required	Real

- Go `http.Transport` with `tls.Config{MinVersion: TLS13 (or 1.2), Certificates: [clientCert], RootCAs: system}`. Never `InsecureSkipVerify`; no fallback to unauthenticated calls.
- Fail closed at startup in Development/Production if cert/key are missing or mismatched (`tls.X509KeyPair` check + expiry check).
- Per-user Teller access token: HTTP Basic (token as username), sent only server-side over that mTLS client. Timeouts, bounded retries, concurrency cap (AGENTS.md §17).
- Browser only loads Teller Connect with the *application id*; the enrollment token is posted to our API, never kept in the browser.

3. Task sequence (all Banking Agent, separate review each)

Task	Deliverable	Gate
VP-009	Teller adapter + mTLS client, env config, fake-server TLS tests	Owner confirms cert/key location and prod secret source
VP-010	Banking migration (`institution_connections`), enroll/list/disconnect API, credential-at-rest, two-user isolation tests	Owner approves credential encryption mechanism
VP-011	Accounts table + normalized account list API (integer minor units, currency)	Balances in first scope? refresh cadence?
Later	Transactions, sync worker, pending→posted, webhooks, Dashboard cards	Separate tasks

4. Security and data rules

- `user_id` from verified Foundation context only; RLS on every Banking table; Banking owns its migrations.
- Teller tokens: encrypted at rest (e.g. pgsodium/Vault or app-level AES-GCM with key outside DB) — **DECIDE**; never in responses or logs; redact in error paths.
- Money: parse Teller decimal strings to integer minor units; never float.
- Disconnect: revoke at Teller, delete local credential, keep minimal audit row if approved.
- Provider failure keeps last synced data and shows freshness/error; must not affect Receipts.

5. Open decisions

1. **DECIDE** cert/key local path and production secret store.
2. **DECIDE** credential-at-rest mechanism.
3. **DECIDE** VP-009 scope: client only, or plus a connection probe.
4. **DECIDE** balances in VP-011; manual refresh vs scheduled sync.
5. Confirm Plaid stays out of scope (Teller only) for this phase.
6. Confirm Sandbox for all UI/CI; Development manual smoke only.

Not verified: current Teller doc details (re-check teller.io/docs before VP-009), and whether the cert is already issued from the Teller dashboard.